

Skill - Week11

Session 21:

```
GNU nano 8.7.1                                jobs.c *
#include "jobs.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <errno.h>

static Job jobs[MAX_JOBS];
static int job_count = 0;
static pid_t shell_pgid;
static int terminal_fd = STDIN_FILENO;

/* Initialize shell signal actions */
void init_shell(void)
{
    shell_pgid = getpgrp();

    signal(SIGINT, SIG_IGN);
    signal(SIGQUIT, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);

    printf("Shell PID: %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);
}

/* Add a new job */
static void add_job(pid_t pid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].pgid = pid;
        jobs[job_count].status = RUNNING;
        job_count++;
    }
}
```

```
#include <stdio.h>
#include "jobs.h"
int main(void)
{
    int choice;
    init_shell();
    while (1)
    {
        printf("\n==== MINI JOB CONTROL SHELL ==== \n");
        printf("1. Create Foreground Job\n");
        printf("2. Show Job Table\n");
        printf("3. Resume Stopped Job\n");
        printf("4. Terminate Job\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input.\n");
            break;
        }
        switch (choice)
        {
            case 1:
                create_job();
                break;
            case 2:
                show_jobs();
                break;
            case 3:
                resume_job();
                break;
            case 4:
                terminate_job();
                break;
            case 5:
                cleanup_jobs();
                printf("Shell exited safely.\n");
        }
    }
}
```

```

root@DESKTOP-QU0HVJI:~/SKILL/skill-11/job_control_project# gcc -D_GNU_SOURCE -Wall -Wextra
-std=c11 jobs.c main.c -o job_control
root@DESKTOP-QU0HVJI:~/SKILL/skill-11/job_control_project# ./job_control
Shell PID: 2102
Shell PGID: 2102

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Exit
Enter choice: 1
Foreground job started. PID: 2104
Press Ctrl+C to terminate or Ctrl+Z to suspend.
Child 2104 started.
Child 2104 running: 1
Child 2104 running: 2
Child 2104 running: 3
Child 2104 running: 4
Child 2104 running: 5
Child 2104 running: 6
Child 2104 running: 7
^ZJob suspended.

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Exit
Enter choice: 2

--- JOB TABLE ---
Job 1 | PID: 2104 | PGID: 2104 | Stopped

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Exit
Enter choice: |

```

Report: The modules were integrated using a shared header file and separate source files. End-to-end testing verified process creation, signal handling, terminal control, job-state updates, suspension, resumption, error handling, and cleanup.

1. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
GNU nano 8.7.1 errors.c *
#include "errors.h"

#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <stdlib.h>

/* Log an application error */
void log_error(const char *message)
{
    FILE *file = fopen("error.log", "a");

    if (file == NULL)
    {
        perror("Unable to open error.log");
        return;
    }

    time_t now = time(NULL);
    char *timestamp = ctime(&now);

    if (timestamp != NULL)
    {
        timestamp[strcspn(timestamp, "\n")] = '\0';

        fprintf(file, "[%s] ERROR: %s\n",
                timestamp, message);
    }
    else
    {
        fprintf(file, "ERROR: %s\n", message);
    }

    fclose(file);

    fprintf(stderr, "Error: %s\n", message);
}
```

```
GNU nano 0.7.1 main.c
#include <stdio.h>
#include "jobs.h"
#include "errors.h"

int main(void)
{
    int choice;
    int result;

    init_shell();

    while (1)
    {
        printf("\n===== MINI JOB CONTROL SHELL =====\n");
        printf("1. Create Foreground Job\n");
        printf("2. Show Job Table\n");
        printf("3. Resume Stopped Job\n");
        printf("4. Terminate Job\n");
        printf("5. Test Invalid Input\n");
        printf("6. Exit\n");

        result = read_choice(&choice);

        if (result == 0)
        {
            printf("\nInput closed. Cleaning up.\n");
            break;
        }

        if (result == -1)
        {
            printf("Please try again.\n");
            continue;
        }

        switch (choice)
        {

```

Output:

```
root@DESKTOP-QU0HVJI:~/job_control_project$ nano Makefile
root@DESKTOP-QU0HVJI:~/job_control_project$ make clean
make
./job_control
rm -f job_control
gcc -Wall -Wextra main.c jobs.c errors.c -o job_control
Shell PID: 800
Shell PGID: 800

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Test Invalid Input
6. Exit
Enter choice: 2

--- JOB TABLE ---

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Test Invalid Input
6. Exit
Enter choice: 1
Foreground job started. PID: 807
Press Ctrl+C to terminate or Ctrl+Z to suspend.
Child 807 running: 1
Child 807 running: 2
Child 807 running: 3
Child 807 running: 4
Child 807 running: 5
Child 807 running: 6
^CJob completed or terminated.

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Test Invalid Input
```

Report: The mini shell was tested using gcc -Wall -Wextra, ./job_control, and cat error.log.

Session 22:

1. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
GNU nano 8.7.1                                memory1.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(5 * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
        printf("%d ", ptr[i]);
    }

    printf("\n");

    return 0;
}
```

```
Shell exited safely.
root@DESKTOP-QU0HVJI:~/SKILL/skill-11/job_control_project# cd job_control_project
-bash: cd: job_control_project: No such file or directory
root@DESKTOP-QU0HVJI:~/SKILL/skill-11/job_control_project# cd ..
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# nano memory1.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# gcc -Wall -Wextra memory1.c -o memory1
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# ./memory1
1 2 3 4 5
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# valgrind --leak-check=full --show-leak-kinds=all ./memory1
Command 'valgrind' not found, but can be installed with:
snap install valgrind # version 3.26.0, or
apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# |
```

Report: The C program was compiled using gcc -g -Wall -Wextra and tested using ./memory. Valgrind detected the memory leak caused by missing free().

2. To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
GNU nano 8.7.1 calculator.c *
#include <stdio.h>

int main() {
    int a, b;

    scanf("%d %d", &a, &b);

    printf("%d\n", a + b);

    return 0;
}

root@DESKTOP-QU0HVJI:~/SKILL/skill-11# nano calculator.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# gcc -Wall -Wextra calculator.c -o calculator
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# ./calculator
^C
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# nano test.sh
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# chmod +x test.sh
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# ./test.sh
TC1: Expected=8 Actual=8 PASS
TC2: Expected=30 Actual=30 PASS
TC3: Expected=0 Actual=0 PASS
TC4: Expected=0 Actual=0 PASS
Test report generated: report.csv
root@DESKTOP-QU0HVJI:~/SKILL/skill-11# |
```

Report: Test cases were designed and executed automatically using a Bash script. The program was compiled using gcc, and its outputs were compared with expected results. All four test cases passed, and the results were saved in report.csv using cat report.csv for verification.